

Programmation Web Frontend

Session 5 - Version 3.0

L2/X64/06/PWF/187628/26-27

1. Les bases du Web dynamique



1. Les bases du Web dynamique

- ✓ Le Web et JavaScript
- ✓ L'environnement, la console
- ✓ Les expressions
- ✓ Les variables et leurs types
- ✓ La conversion de types
- ✓ JavaScript dans la page
- ✓ Kits 1 et 2 : écrire, lire, cliquer

- ✓ Les opérateurs, les conditions

Les tableaux

Les boucles

Chaînes et nombres

Les fonctions, la portée

Les objets

Le portfolio dans la page

Le dépôt du cours

<https://github.com/eloudsa/ifapme-x64-pwf>

Les slides

`slides/`

Le PDF de chaque session

Pour relire à votre rythme

Les labos

`session-5/labo-1/`

La page de départ : `index.html`,
`style.css`, `main.js`

Disponibles avant la session

Les solutions

`session-5/labo-1/solutions/labo-1-1.js`

Une solution par niveau

Publiées après la session

Un seul endroit pour tout retrouver : slides, labos et solutions

Les commandes git

- **git clone** : une seule fois, crée le dossier **ifapme-x64-pwf**
- **git pull** : à chaque session, récupère les nouveautés
- Après la session, un nouveau **git pull** apporte les solutions
- Copiez le dossier du labo hors du clone : vous travaillez sur cette copie

```
# Une seule fois : copier le dépôt sur votre ordinateur  
git clone https://github.com/eloudsa/ifapme-x64-pwf.git
```

```
# À chaque session : récupérer les nouveautés  
cd ifapme-x64-pwf  
git pull
```

Votre kit de survie

Étape	Instructions	Vue en
Kit 1	<script defer>, querySelector, textContent	Session 3
Kit 2	const, .value, addEventListener("click")	Session 3
Kit 3	classList, style, checked	Session 4

Les tableaux

Pourquoi un tableau ?

Une variable par valeur

```
const color1 = "#4A72B8"  
const color2 = "#F2994A"  
const color3 = "#2E9E5B"  
const color4 = "#C0392B"  
// et la cinquième ?
```

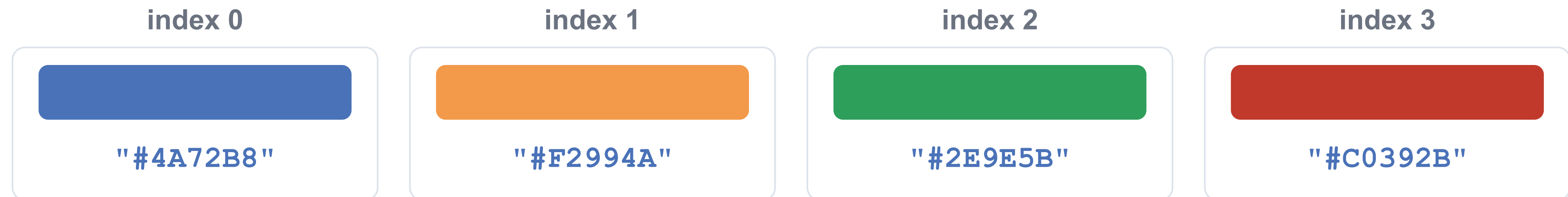
Un tableau

```
const colors = [  
  "#4A72B8",  
  "#F2994A",  
  "#2E9E5B",  
  "#C0392B",  
]
```

- Une seule variable, plusieurs valeurs, **dans un ordre précis**
- La liste peut grandir ou rétrécir pendant l'exécution

Qu'est-ce qu'un tableau ?

Chaque valeur occupe une position numérotée : l'index



```
const colors = ["#4A72B8", "#F2994A", "#2E9E5B", "#C0392B"]  
console.log(colors[0])           // #4A72B8  
console.log(colors.length)       // 4
```

Le premier élément est à l'index **0**, pas 1

Lire un élément

- `pages[0]` : le premier
- `pages[pages.length - 1]` : le dernier
- `pages.length` : le nombre d'éléments
- Index inexistant : `undefined`, pas d'erreur

```
const pages = ["Accueil", "Projets", "Blog", "Contact"]  
  
console.log(pages[0])  
console.log(pages[pages.length - 1])  
console.log(pages.length)  
console.log(pages[7])
```

```
Accueil  
Contact  
4  
undefined
```

CONSOLE

Remplacer un élément

- `pages[2] = "Journal"` : une case change
- Le tableau garde sa taille
- Puis on affiche, comme d'habitude

```
const pages = ["Accueil", "Projets", "Blog", "Contact"]  
pages[2] = "Journal"
```

```
const info = document.querySelector("#info")  
info.textContent = `Rubrique 3 : ${pages[2]}`
```

Mon site

Rubrique 3 : Journal

APERÇU

const et les tableaux

const protège la boîte, pas son contenu

```
const tags = ["ux", "mobile"]

tags[0] = "ui"           // permis : on change le contenu
console.log(tags)

tags = ["figma"]         // interdit : on remplace la boîte
```

```
(2) ['ui', 'mobile']
Uncaught TypeError:
Assignment to constant
variable.
```

CONSOLE

Déclarez vos tableaux avec **const** : ils peuvent grandir, rétrécir et changer

Kit 4 : afficher une liste

Kit de survie 4

Instruction	Ce qu'elle fait	En détail
<code>el.innerHTML = `<li>...</li>`</code>	Écrire du HTML dans un élément	S10, S12
<code>tableau.join("")</code>	Assembler les morceaux de HTML d'une liste	S5

Une recette : utilisez-la telle quelle, chaque pièce sera expliquée en détail plus tard

textContent ou innerHTML ?

- **textContent** : du texte, affiché tel quel
- **innerHTML** : du HTML, interprété
- Les balises deviennent de vrais éléments

```
const a = document.querySelector("#a")
const b = document.querySelector("#b")

a.textContent = "<strong>Figma</strong>"
b.innerHTML = "<strong>Figma</strong>"
```

#a avec textContent :

Figma

#b avec innerHTML :

Figma

APERÇU

innerHTML et template literal

- Un template literal : plusieurs lignes, des `${...}`
- Idéal pour écrire un petit bloc de HTML
- Le contenu précédent est remplacé

```
const name = "App resto"  
const year = 2026  
const card = document.querySelector("#card")  
  
card.innerHTML = `

## ${name}</h2> <p>Projet réalisé en ${year}</p>`


```

App resto

Projet réalisé en 2026

APERÇU

join : assembler un tableau

- Une seule chaîne, un séparateur au choix
- Le séparateur va **entre** les éléments
- `join("")` : collés, sans séparateur

```
const pages = ["Accueil", "Projets", "Contact"]
```

```
console.log(pages.join(", "))  
console.log(pages.join(" · "))  
console.log(pages.join(""))  
console.log(pages.join("</li><li>"))
```

```
Accueil, Projets, Contact  
Accueil · Projets · Contact  
AccueilProjetsContact  
Accueil</li><li>Projets</li><li>Contact
```

CONSOLE

La recette de la liste

```
"<li>" + tableau.join("</li><li>") + "</li>"
```

1. Le tableau

```
["A", "B", "C"]
```

2. join

Entre chaque élément : `</li><li>`

```
A</li><li>B</li><li>C
```

3. Les bords

"" devant, "" derrière

```
<li>A</li><li>B</li><li>C</li>
```

join ne met rien avant le premier ni après le dernier : on ajoute les bords à la main

Une liste dans la page

- Dans le HTML : un `<ul id="tools">` vide
- Le tableau est la source
- Une ligne pour tout afficher

```
const tools = ["Figma", "FigJam", "Notion"]
const list = document.querySelector("#tools")

list.innerHTML = "<li>" + tools.join("</li><li>") + "</li>"
```

Mes outils

- Figma
- FigJam
- Notion

APERÇU

Labo 1 : la liste d'envies

Niveau 1 · Simple

En chiffres

- Un tableau `wishes` de quatre envies
- Leur nombre dans `#count`
- La première et la dernière

Niveau 2 · Modéré

La liste affichée

- Remplacer l'envie à l'index 1
- Toute la liste dans `#list`
- Un index qui n'existe pas

Niveau 3 · Avancé

Remplacer au clic

- Le champ remplace la première envie
- Réafficher la liste
- Champ vide : un message
- Et `wishes = [...]` ?

Labo 1 · niveau 1

Niveau 1 · Simple

En chiffres

- Créez un tableau `wishes` de quatre envies (matériel, logiciel, formation...)
- Affichez dans `#count` : « 4 envies dans la liste »
- Affichez la première envie dans `#first`, la dernière dans `#last`

Hint

La dernière :

`wishes[wishes.length - 1]`,
jamais un nombre écrit en dur

Labo 1 · niveau 2

Niveau 2 · Modéré

La liste affichée

- Remplacez l'envie à l'index 1, avant tout affichage
- Affichez toute la liste dans `#list` avec la recette du kit 4
- Affichez `wishes[10]` dans la console : que se passe-t-il ?

Hint

```
"<li>" + wishes.join("</li><li>") + "</li>"
```


Labo 1 · niveau 3

Niveau 3 · Avancé

Remplacer au clic

- Au clic sur `#replace`, le texte du champ `#wish` remplace la première envie
- Réaffichez la liste et `#first`, puis videz le champ
- Champ vide : un message dans `#message`, aucun remplacement
- Essayez `wishes = ["Rien"]` : que dit la console ?

Hint

Le tableau change, la page non : réécrivez la ligne d'affichage dans le handler

Solution labo 1 · niveau 1

Niveau 1 · Simple

```
const wishes = [
  "Tablette graphique",
  "Nuancier Pantone",
  "Licence Figma",
  "Écran calibré",
]

const count = document.querySelector("#count")
const first = document.querySelector("#first")
const last = document.querySelector("#last")

count.textContent = `${wishes.length} envies dans la liste`
first.textContent = `Première : ${wishes[0]}`
last.textContent = `Dernière : ${wishes[wishes.length - 1]}`
console.log(wishes)
```



Ma liste d'envies

4 envies dans la liste

Première : Tablette graphique

Dernière : Écran calibré

APERÇU

Solution labo 1 · niveau 2

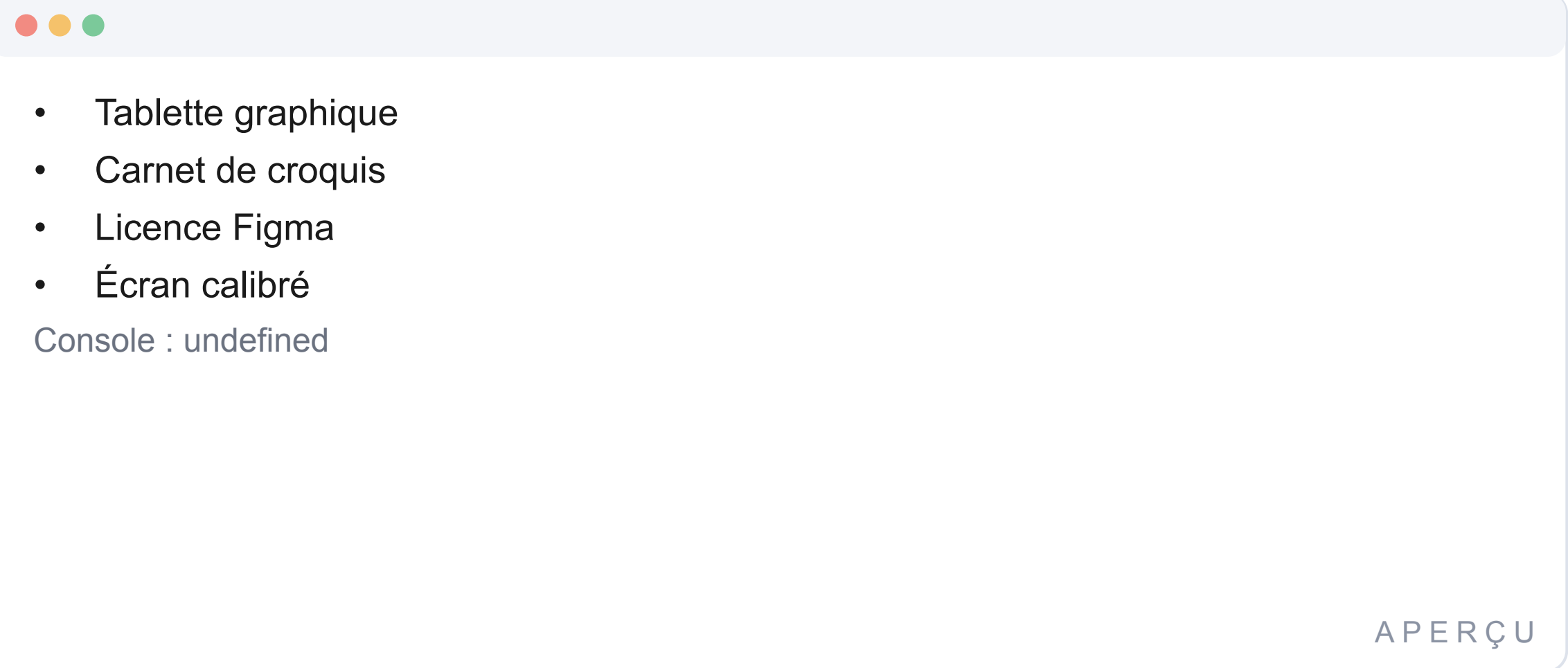
Niveau 2 · Modéré

```
const wishes = [
  "Tablette graphique",
  "Nuancier Pantone",
  "Licence Figma",
  "Écran calibré",
]
wishes[1] = "Carnet de croquis"
```

```
const count = document.querySelector("#count")
const first = document.querySelector("#first")
const last = document.querySelector("#last")
const list = document.querySelector("#list")

count.textContent = `${wishes.length} envies dans la liste`
first.textContent = `Première : ${wishes[0]}`
last.textContent = `Dernière : ${wishes[wishes.length - 1]}`
list.innerHTML = "<li>" + wishes.join("</li><li>") + "</li>"

console.log(wishes[10])
```



• Tablette graphique

• Carnet de croquis

• Licence Figma

• Écran calibré

Console : undefined

APERÇU

Solution labo 1 · niveau 3

Niveau 3 · Avancé

```
// ... code du niveau 2, inchangé
const wishInput = document.querySelector("#wish")
const replaceButton = document.querySelector("#replace")
const message = document.querySelector("#message")
```

```
replaceButton.addEventListener("click", () => {
  if (wishInput.value === "") {
    message.textContent = "Écrivez d'abord une envie"
  } else {
    wishes[0] = wishInput.value
    list.innerHTML = "<li>" + wishes.join("</li><li>") + "</li>"
    first.textContent = `Première : ${wishes[0]}`
    message.textContent = ""
    wishInput.value = ""
  }
})

// wishes = ["Rien"] → TypeError
```

- Stylet
- Carnet de croquis
- Licence Figma
- Écran calibré

Remplacer la première

Après « Stylet » : la première envie est remplacée

APERÇU

Ajouter, retirer, chercher

Ajouter et retirer

Quatre méthodes, deux extrémités. Elles modifient le tableau

Méthode	Avant → après	Ce qu'elle fait
<code>push(x)</code>	<code>[a, b] → [a, b, x]</code>	Ajoute à la fin
<code>pop()</code>	<code>[a, b, c] → [a, b]</code>	Retire le dernier et le renvoie
<code>unshift(x)</code>	<code>[a, b] → [x, a, b]</code>	Ajoute au début
<code>shift()</code>	<code>[a, b, c] → [b, c]</code>	Retire le premier et le renvoie

Le plus utilisé : **push**. Les retraits renvoient l'élément retiré

Garder ce qui sort

- `pop()` et `shift()` renvoient l'élément retiré
- Rangez-le dans une `const`
- Sinon, il est perdu

```
const layers = ["Fond", "Image", "Titre"]  
layers.push("Bouton")
```

```
const removed = layers.pop()  
console.log(`Calque retiré : ${removed}`)  
console.log(layers)
```

```
Calque retiré : Bouton  
(3) ['Fond', 'Image', 'Titre']
```

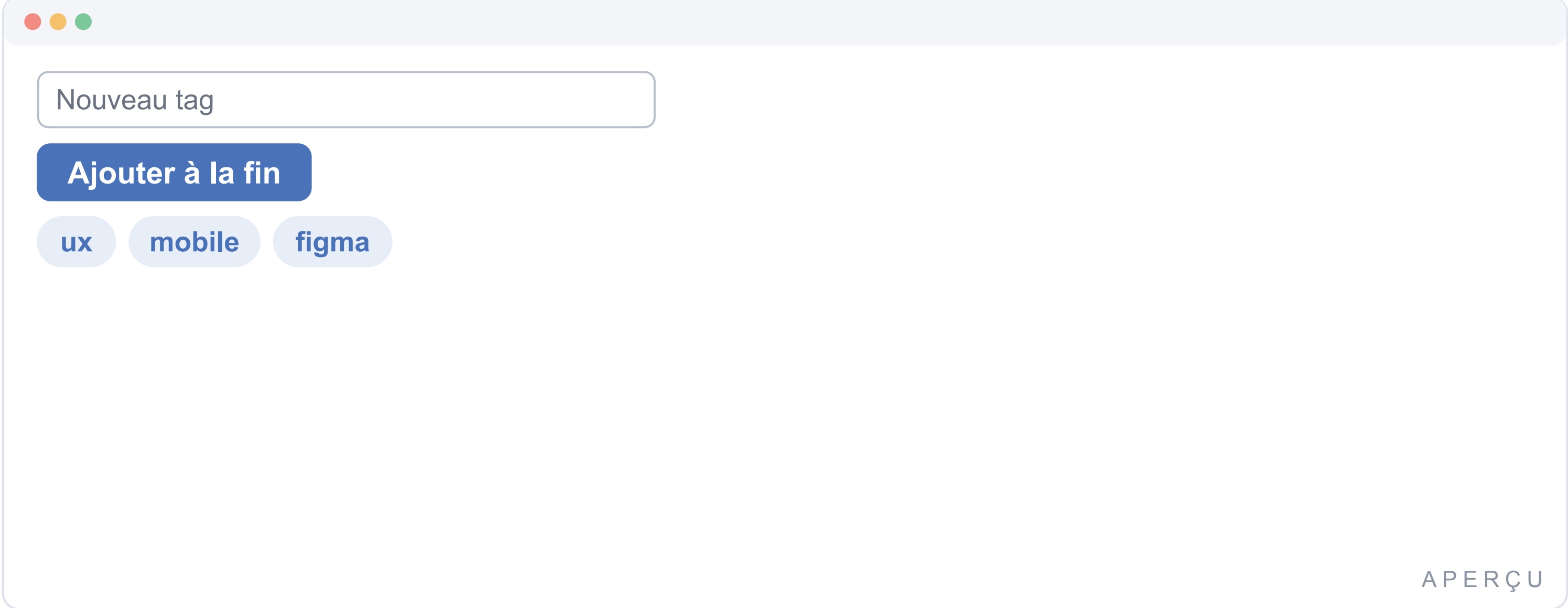
CONSOLE

Ajouter au clic

- Le tableau change : **push**
- Puis on réaffiche : la recette
- **champ.value = ""** vide le champ

```
const tags = ["ux", "mobile"]

addButton.addEventListener("click", () => {
  tags.push(tagInput.value)
  tagList.innerHTML = "<li>" + tags.join("</li><li>") + "</li>"
  tagInput.value = ""
})
```



The screenshot shows a web application interface with a light gray background. At the top, there is a text input field with the placeholder text "Nouveau tag". Below the input field is a blue button with the text "Ajouter à la fin". Underneath the button, there are three rounded rectangular tags: "ux", "mobile", and "figma". In the bottom right corner of the interface, the word "APERÇU" is displayed in a small, gray, sans-serif font.

Chercher : includes et indexOf

- `includes(x)` : présent ? `true` ou `false`
- `indexOf(x)` : à quelle position ?
- Absent : `indexOf` renvoie `-1`
- Comparaison stricte, comme `===`

```
const tags = ["ux", "ui", "figma"]

console.log(tags.includes("figma"))
console.log(tags.includes("sketch"))
console.log(tags.indexOf("ui"))
console.log(tags.indexOf("sketch"))
console.log(tags.includes("UX"))
```

```
true
false
1
-1
false
```

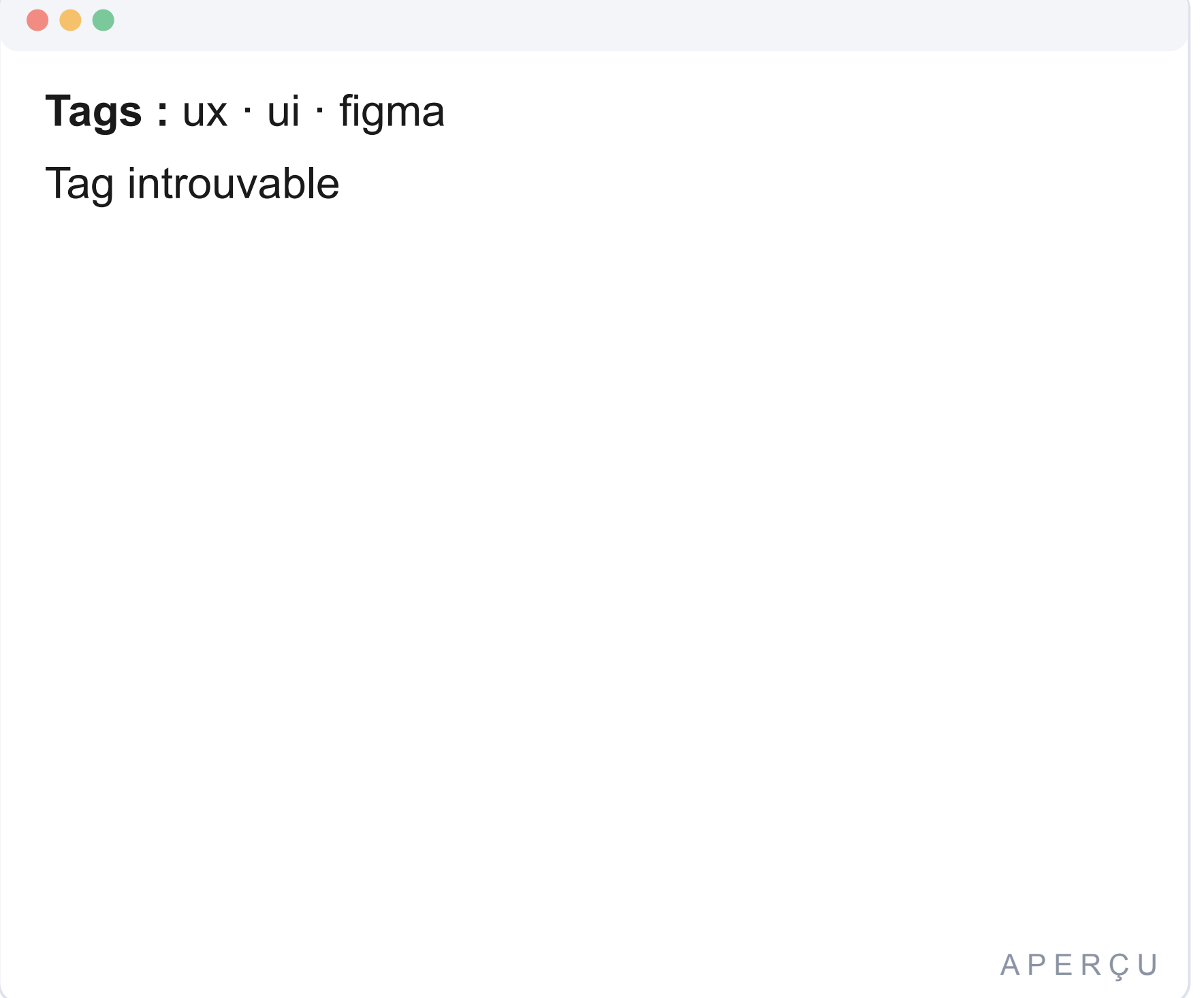
CONSOLE

Le résultat -1

-1 n'est ni une erreur, ni false : c'est un nombre

```
const tags = ["ux", "ui", "figma"]
const index = tags.indexOf("sketch")

if (index === -1) {
  message.textContent = "Tag introuvable"
} else {
  message.textContent = `Position : ${index + 1}`
}
```



Testez toujours le -1 avant d'utiliser une position

Labo 2 : ajouter et retirer

Niveau 1 · Simple

La fin de la liste

- Des tags affichés en pastilles
- « Ajouter à la fin » : `push`
- « Retirer le dernier » : `pop`

Niveau 2 · Modéré

Le début de la liste

- « Ajouter au début » : `unshift`
- « Retirer le premier » : `shift`
- Un compteur « 3 tags »

Niveau 3 · Avancé

Refuser et prévenir

- Champ vide : refusé
- Doubleton : refusé, avec sa position
- Liste vide : un message

Labo 2 · niveau 1

Niveau 1 · Simple

La fin de la liste

- Un tableau `tags` avec `"ux"` et `"mobile"`, affiché dans `#tags`
- Clic sur `#add` : le texte de `#tag` s'ajoute à la fin, la liste se réaffiche, le champ se vide
- Clic sur `#remove-last` : le dernier tag disparaît, son nom s'affiche dans `#message`

Hint

`pop()` renvoie le tag retiré :
rangez-le dans une `const` avant
d'afficher

Labo 2 · niveau 2

Niveau 2 · Modéré

Le début de la liste

- Clic sur `#add-first` : le tag s'ajoute au début
- Clic sur `#remove-first` : le premier tag disparaît, son nom s'affiche
- Dans `#count` : « 3 tags », au chargement et après chaque clic

Hint

Le nombre de tags ne se compte pas à la main : `tags.length`

Labo 2 · niveau 3

Niveau 3 · Avancé

Refuser et prévenir

- « Ajouter à la fin » avec un champ vide : un message, aucun ajout
- Tag déjà présent : « ux : déjà en position 1 », aucun ajout
- « Retirer le dernier » sur une liste vide : « Plus rien à retirer »

Hint

Trois cas : `if`, `else if`, `else`.
Comparez `value` à `""`, puis
`includes`

Solution labo 2 · niveau 1

Niveau 1 · Simple

```
const tags = ["ux", "mobile"]

const tagInput = document.querySelector("#tag")
const tagList = document.querySelector("#tags")
const message = document.querySelector("#message")
const addButton = document.querySelector("#add")
const removeLastButton = document.querySelector("#remove-last")

tagList.innerHTML = "<li>" + tags.join("</li><li>") + "</li>"
```

```
addButton.addEventListener("click", () => {
  tags.push(tagInput.value)
  tagList.innerHTML = "<li>" + tags.join("</li><li>") + "</li>"
  tagInput.value = ""
  console.log(tags)
})

removeLastButton.addEventListener("click", () => {
  const removed = tags.pop()
  tagList.innerHTML = "<li>" + tags.join("</li><li>") + "</li>"
  message.textContent = `Tag retiré : ${removed}`
})
```



Nouveau tag

Ajouter à la fin

Retirer le dernier

ux

Tag retiré : mobile

Après : figma ajouté, puis deux retraits

APERÇU

Solution labo 2 · niveau 2

Niveau 2 · Modéré

```
// + la ligne count dans les handlers du niveau 1
addFirstButton.addEventListener("click", () => {
  tags.unshift(tagInput.value)
  tagList.innerHTML = "<li>" + tags.join("</li><li>") + "</li>"
  count.textContent = `${tags.length} tags`
  tagInput.value = ""
})
```

```
removeFirstButton.addEventListener("click", () => {
  const removed = tags.shift()
  tagList.innerHTML = "<li>" + tags.join("</li><li>") + "</li>"
  count.textContent = `${tags.length} tags`
  message.textContent = `Tag retiré : ${removed}`
})
```



Ajouter au début

Retirer le premier

ux

mobile

figma

3 tags

Tag retiré : resto

APERÇU

Solution labo 2 · niveau 3

Niveau 3 · Avancé

```
// ... handlers du début : ceux du niveau 2
addButton.addEventListener("click", () => {
  const tag = tagInput.value
  if (tag === "") {
    message.textContent = "Écrivez d'abord un tag"
  } else if (tags.includes(tag)) {
    const position = tags.indexOf(tag) + 1
    message.textContent = `${tag} : déjà en position ${position}`
  } else {
    tags.push(tag)
    tagList.innerHTML = "<li>" + tags.join("</li><li>") + "</li>"
    count.textContent = `${tags.length} tags`
    message.textContent = `${tag} ajouté`
  }
  tagInput.value = ""
})
```

```
removeLastButton.addEventListener("click", () => {
  if (tags.length === 0) {
    message.textContent = "Plus rien à retirer"
  } else {
    const removed = tags.pop()
    tagList.innerHTML = "<li>" + tags.join("</li><li>") + "</li>"
    count.textContent = `${tags.length} tags`
    message.textContent = `Tag retiré : ${removed}`
  }
})
```



Nouveau tag

Ajouter à la fin

ux : déjà en position 1

Après quatre retraits :

0 tags

Plus rien à retirer

APERÇU

Extraire et modifier

slice : copier un morceau

- `slice(début, fin)` : la fin est **exclue**
- Sans fin : jusqu'au bout
- L'original n'est pas modifié
- `slice()` : une copie complète

```
const pages = ["Accueil", "Projets", "Blog", "Contact"]  
  
console.log(pages.slice(0, 2))  
console.log(pages.slice(2))  
console.log(pages)
```

```
(2) ['Accueil', 'Projets']  
(2) ['Blog', 'Contact']  
(4) ['Accueil', 'Projets', 'Blog', 'Contact']
```

CONSOLE

splice : modifier sur place

- `splice(index, combien)` : retire
- `splice(index, 0, x)` : insère
- Renvoie un **tableau** des éléments retirés
- L'original est modifié

```
const pages = ["Accueil", "Projets", "Blog", "Contact"]

const removed = pages.splice(2, 1)
pages.splice(1, 0, "Studio")

console.log(removed)
console.log(pages)
```

```
['Blog']
(4) ['Accueil', 'Studio', 'Projets', 'Contact']
```

CONSOLE

slice ou splice ?

Une lettre de différence, deux métiers

slice

```
pages.slice(0, 3)
```

Copie un morceau

L'original est intact

splice

```
pages.splice(2, 1)
```

Retire ou insère

L'original est modifié

Le piège

```
pages.splice(-1, 1)
```

Retire le **dernier** élément

indexOf a renvoyé -1 ? Testez avant

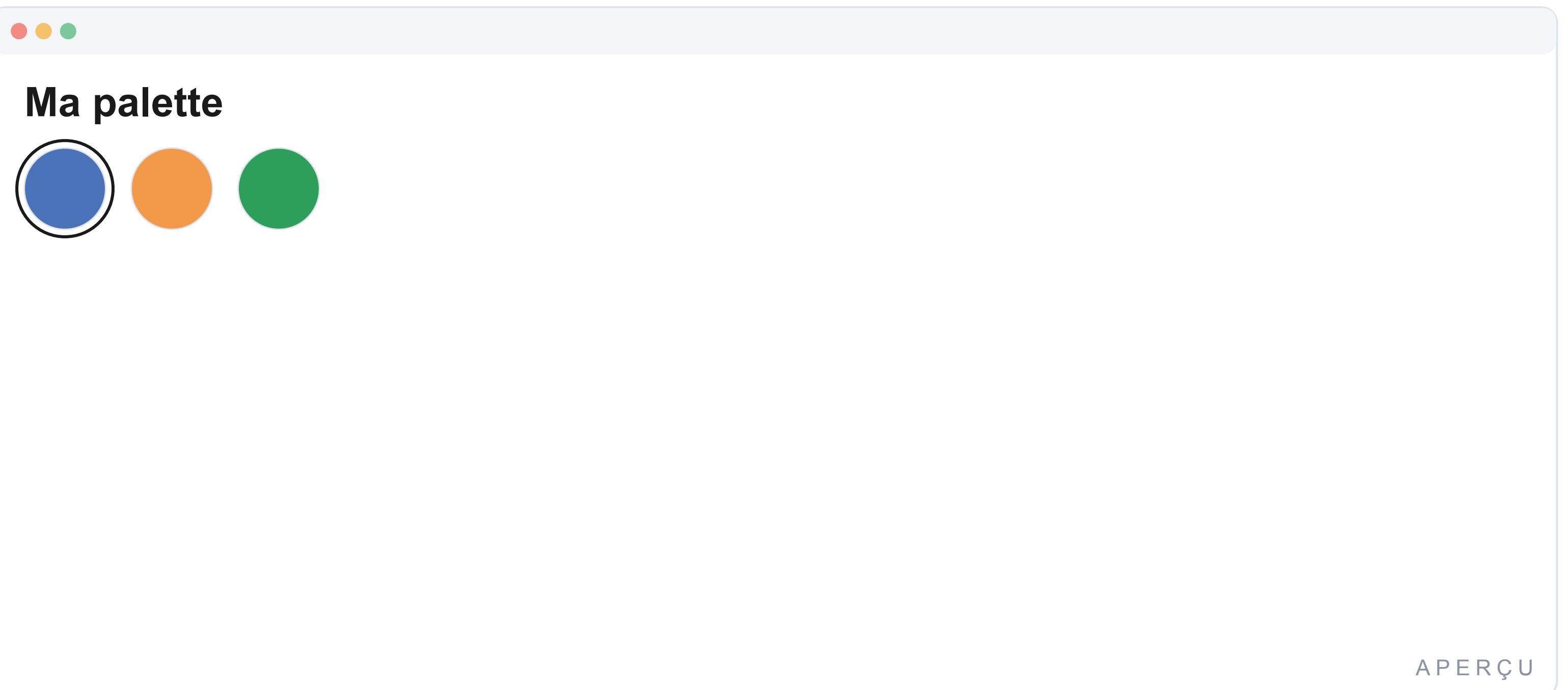
Une position qui vient d'`indexOf` se teste toujours avant d'aller dans `splice`

Des pastilles avec join

- Même idée que la liste
- **open** : le début d'une pastille
- **close** : sa fin
- join colle **close** + **open** entre les couleurs

```
const palette = ["#4A72B8", "#F2994A", "#2E9E5B"]
const open = `<span class="swatch" style="background: `
const close = `"></span>`

swatches.innerHTML = open + palette.join(close + open) + close
```



Projet : le gestionnaire de palette

- Une pastille par couleur
- Les trois premières : les **principales**
- 5 couleurs au plus, pas de doublon
- Palette pleine : la plus ancienne sort
- La favorite passe en tête

```
const palette = ["#4A72B8", "#F2994A", "#2E9E5B", "#C0392B"]  
const maxColors = 5
```

Ma palette

Couleurs principales : #4A72B8 · #F2994A · #2E9E5B

Ajouter

Mettre en tête

Retirer

APERÇU

Labo 3 : le gestionnaire de palette

Niveau 1 · Simple

La palette affichée

- Quatre pastilles dans `#palette`
- Les principales avec `slice`
- « Ajouter » : `push`

Niveau 2 · Modéré

Les règles

- Champ vide : refusé
- Doublon : refusé
- Pleine : la plus ancienne sort
- Un message à chaque fois

Niveau 3 · Avancé

Le favori en tête

- `indexOf`, `splice`, `unshift`
- Absente ou déjà en tête : un message
- « Retirer » sans erreur de cible

Labo 3 · niveau 1

Niveau 1 · Simple

La palette affichée

- Un tableau `palette` de quatre couleurs hexadécimales
- Une pastille par couleur dans `#palette`
- Dans `#main` : les trois premières, séparées par « · »
- Clic sur `#add` : la couleur du champ `#color` s'ajoute, tout se réaffiche

Hint

```
palette.slice(0,  
3).join(" · ") : on peut  
enchaîner deux méthodes
```

Labo 3 · niveau 2

Niveau 2 · Modéré

Les règles

- Champ vide : un message dans `#message`, aucun ajout
- Couleur déjà présente : un message, aucun ajout
- Palette pleine (`maxColors = 5`) : la plus ancienne est retirée, avec un message
- Sinon : ajout, et une confirmation

Hint

Dans l'ordre : est-elle vide ? Déjà là ? Reste-t-il de la place ?

Labo 3 · niveau 3

Niveau 3 · Avancé

Le favori en tête

- Clic sur `#favorite` : la couleur du champ passe en tête
- Absente ou déjà en tête : un message, rien ne bouge
- Clic sur `#remove` : la couleur du champ est retirée
- Couleur introuvable : aucune autre couleur ne doit disparaître

Hint

`indexOf` vaut -1, 0 ou plus : quels cas demandent une action ?

Solution labo 3 · niveau 1

Niveau 1 · Simple

```
const palette = ["#4A72B8", "#F2994A", "#2E9E5B", "#C0392B"]
const open = `<span class="swatch" style="background: `
const close = `"></span>`

const swatches = document.querySelector("#palette")
const main = document.querySelector("#main")
const colorInput = document.querySelector("#color")
const addButton = document.querySelector("#add")
```

```
swatches.innerHTML = open + palette.join(close + open) + close
main.textContent = palette.slice(0, 3).join(" · ")

addButton.addEventListener("click", () => {
  palette.push(colorInput.value)
  swatches.innerHTML = open + palette.join(close + open) + close
  main.textContent = palette.slice(0, 3).join(" · ")
  colorInput.value = ""
})
```

Principales : #4A72B8 · #F2994A · #2E9E5B



Ajouter

Après l'ajout de #8E44AD

APERÇU

Solution labo 3 · niveau 2

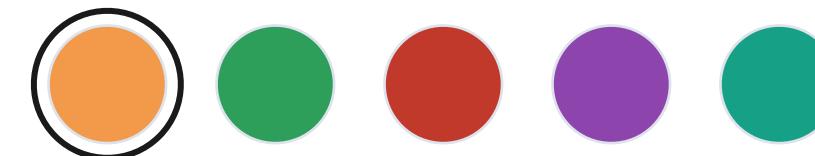
Niveau 2 · Modéré

```
addButton.addEventListener("click", () => {  
  const color = colorInput.value  
  if (color === "") {  
    message.textContent = "Saisissez une couleur"  
  } else if (palette.includes(color)) {  
    message.textContent = `${color} est déjà dans la palette`  
  } else {  
    if (palette.length >= maxColors) {  
      const oldest = palette.shift()  
      message.textContent = `Palette pleine : ${oldest} retirée`  
    } else {  
      message.textContent = `${color} ajoutée`  
      palette.push(color)  
      swatches.innerHTML = open + palette.join(close + open) + close  
      main.textContent = palette.slice(0, 3).join(" · ")  
      colorInput.value = ""  
    }  
  }  
})
```

```
message.textContent = `Palette pleine : ${oldest} retirée`  
} else {  
  message.textContent = `${color} ajoutée`  
}  
palette.push(color)  
swatches.innerHTML = open + palette.join(close + open) + close  
main.textContent = palette.slice(0, 3).join(" · ")  
colorInput.value = ""  
})
```



Principales : #F2994A · #2E9E5B · #C0392B



Palette pleine : #4A72B8 retirée

Après #8E44AD, #8E44AD à nouveau, puis #16A085

APERÇU

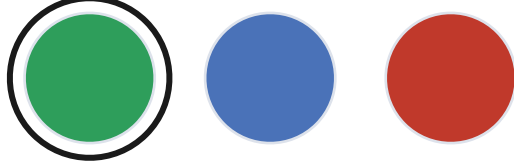
Solution labo 3 · niveau 3

Niveau 3 · Avancé

```
// ... niveaux 1 et 2 inchangés
favoriteButton.addEventListener("click", () => {
  const color = colorInput.value
  const index = palette.indexOf(color)
  if (index === -1) {
    message.textContent = `${color} n'est pas dans la palette`
  } else if (index === 0) {
    message.textContent = `${color} est déjà en tête`
  } else {
    palette.splice(index, 1)
    palette.unshift(color)
    swatches.innerHTML = open + palette.join(close + open) + close
    main.textContent = palette.slice(0, 3).join(" · ")
    message.textContent = `${color} passe en tête`
  }
})
```

```
removeButton.addEventListener("click", () => {
  const index = palette.indexOf(colorInput.value)
  if (index === -1) {
    message.textContent = "Couleur introuvable"
  } else {
    const removed = palette.splice(index, 1)
    swatches.innerHTML = open + palette.join(close + open) + close
    main.textContent = palette.slice(0, 3).join(" · ")
    message.textContent = `${removed[0]} retirée`
  }
})
```

Principales : #2E9E5B · #4A72B8 · #C0392B



#F2994A retirée

Après : #000000 en tête (refusé), #2E9E5B en tête, #F2994A retirée

APERÇU

En résumé

Tableau

Plusieurs valeurs
ordonnées, entre crochets

Index

On compte à partir de 0 ; le
dernier : `length - 1`

const

Protège la boîte, pas son
contenu

Afficher

`innerHTML` et la recette
`join("</li><li>")`

Extrémités

`push` / `pop` à la fin,
`unshift` / `shift` au début

Chercher

`includes` : oui / non ;
`indexOf` : où, ou -1

slice, splice

`slice` copie ; `splice`
modifie sur place

Prochaine session

Les boucles : parcourir un
tableau, élément par
élément